Rail Grade Crossing Monitoring System

Code Reference & Documentation

This document covers the Zone of Interest (ZOI) stabilization module — the foundational layer of the AI-based grade crossing monitoring system. Each section contains the full code for a notebook cell followed by a plain English explanation of what the code does and why it works that way. Additional modules (vehicle detection, safety logic, alerts) will be appended to this document as the project progresses.

Cell 0 — Environment Setup

Code:

```
# Cell 0 - Environment Setup
# Run this cell first if you are setting up for the first time

import subprocess
import sys

required_packages = [
    "opencv-python",
    "numpy",
    "matplotlib",
    "ultralytics"
]

print("Checking required packages...")
for package in required_packages:
    try:
        __import__(package.replace("-", "_"))
        print(f"  ✓ {package}")
    except ImportError:
        print(f"  Installing {package}...")
        subprocess.run([sys.executable, "-m", "pip", "install", package])

print("\nAll packages ready. You can now run the remaining cells.")
```

Explanation:

Cell 0 is a one-time setup cell intended for anyone opening the project for the first time. It iterates over the four required packages — opencv-python, numpy, matplotlib, and ultralytics (which provides YOLO) — and attempts to import each one. If a package is already installed the import succeeds and a checkmark is printed. If the import raises an ImportError, the cell automatically calls pip install using Python's subprocess module to install it in the same environment the notebook is running in. This approach is more reliable than a requirements.txt because it installs directly into the active Python kernel rather than a separate environment. This cell does not need to be re-run on subsequent sessions — only on first setup or if a new dependency is added to the project.

Cell 1 — Imports & Environment Check

Code:

```
# Cell 1 - Imports & Environment Check
import cv2
import numpy as np
import matplotlib.pyplot as plt
import importlib
import pipeline_helpers
importlib.reload(pipeline_helpers)
from pipeline_helpers import (
    stabilize_zoi, build_zoi_mask, apply_mog2_mask,
    check_vehicle_in_zoi, get_debris_mask,
    evaluate_status, draw_frame
)

# Make images display inline in the notebook
%matplotlib inline

# Verify versions
print(f"OpenCV version: {cv2.__version__}")
print(f"NumPy version: {np.__version__}")
print("All imports successful!")
```

Explanation:

Cell 1 brings in the core libraries and helper functions the entire pipeline depends on. OpenCV (cv2) handles all image and video processing. NumPy (np) provides the array operations that OpenCV relies on. Matplotlib (plt) displays images inline in the notebook. The new additions in this cell are the pipeline_helpers module imports: importlib.reload(pipeline_helpers) forces Python to re-read the helper file every time Cell 1 runs, ensuring any edits to pipeline_helpers.py take effect immediately without restarting the kernel. The from pipeline_helpers import statement loads all seven helper functions — stabilize_zoi, build_zoi_mask, apply_mog2_mask, check_vehicle_in_zoi, get_debris_mask, evaluate_status, and draw_frame — directly into the notebook's namespace so they can be called by any cell below. These helpers must be loaded here so they are available to Cells 10, 11, and 12 regardless of cell execution order.

Cell 2 — Load Video and Display First Frame

Code:

```
# Cell 2 - Load video and display first frame
video_path = "../videos/Rail-Grade-Crossing-StressTest.mov" # Change this directory
path for new video test

# Open the video
cap = cv2.VideoCapture(video_path)

# Check it opened correctly
if not cap.isOpened():
    print("Error: Could not open video. Check the filename and path.")
else:
    print(f"Video opened successfully!")
```

```
    print(f"Total frames: {int(cap.get(cv2.CAP_PROP_FRAME_COUNT))}")
    print(f"FPS: {cap.get(cv2.CAP_PROP_FPS)}")
    print(f"Resolution: {int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))} x
{int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))}")

    # Reads the first frame
    ret, frame = cap.read()

    if ret:
        # Converts BGR to RGB
        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

        # Display it
        plt.figure(figsize=(12, 7))
        plt.imshow(frame_rgb)
```

Explanation:

Cell 2 opens the video file using OpenCV's VideoCapture class, which treats the video as a sequence of individual frames that can be read one at a time. The code first checks that the file opened correctly, then reads three key properties: total frame count, frames per second (FPS), and resolution. It then reads the very first frame using cap.read(), which returns two values — a boolean (ret) confirming the read was successful, and the actual frame as a NumPy array of pixel values. Because OpenCV stores color images in BGR order (Blue, Green, Red) rather than the standard RGB order that matplotlib expects, the frame is converted using cvtColor before being displayed. cap.release() is always called at the end to properly close the video file and free up memory.

Cell 3 — Define and Draw the ZOI Polygon

Code:

```
# Cell 3 - Define and Draw the ZOI Polygon on the First Frame
# Coordinate system: X increases right, Y increases down

cap = cv2.VideoCapture(video_path)
ret, frame = cap.read()
cap.release()

# Define the ZOI polygon — adjust points using grid lines as reference
zoi_points = np.array([
    [100, 400],  # TL
    [500, 350],  # TR
    [1650, 480],  # BR
    [1500, 600],  # BL
], dtype=np.int32)

frame_zoi = frame.copy()
h, w = frame_zoi.shape[:2]

# Draw grid lines every 100 pixels for coordinate reference
for x in range(0, w, 100):
    cv2.line(frame_zoi, (x, 0), (x, h), (100, 100, 100), 1)
    cv2.putText(frame_zoi, str(x), (x + 2, 20), cv2.FONT_HERSHEY_SIMPLEX, 0.4,
(255, 255, 0), 1)
for y in range(0, h, 100):
```

```
    cv2.line(frame_zoi, (0, y), (w, y), (100, 100, 100), 1)
    cv2.putText(frame_zoi, str(y), (2, y + 15), cv2.FONT_HERSHEY_SIMPLEX, 0.4,
(255, 255, 0), 1)

# Draw ZOI polygon
cv2.polylines(frame_zoi, [zoi_points], isClosed=True, color=(0, 255, 0),
thickness=5)

# Label each corner with red dot + coordinates
corner_labels = ["TL", "TR", "BR", "BL"]
for pt, label in zip(zoi_points, corner_labels):
    x, y = pt
    cv2.circle(frame_zoi, (x, y), 12, (0, 0, 255), -1)
    cv2.putText(frame_zoi, f"{label} ({x},{y})", (x + 15, y - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)

cv2.putText(frame_zoi, "Zone of Interest (ZOI)", (400, 80),
            cv2.FONT_HERSHEY_SIMPLEX, 2, (0, 255, 0), 4)

frame_rgb = cv2.cvtColor(frame_zoi, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(16, 9))
plt.imshow(frame_rgb)
plt.title("ZOI Polygon — First Frame")
plt.axis("off")
plt.show()
print(f"ZOI Points: {zoi_points.tolist()}")
```

Explanation:

Cell 3 manually defines the Zone of Interest — the danger area on the railroad tracks where a stuck vehicle would be detected. The ZOI is defined as a polygon using four (x, y) coordinate points that correspond to the four corners of the crossing area between the two sets of railroad tracks. In image coordinates, X increases going right and Y increases going downward, so the top-left corner has a smaller Y value than the bottom-left corner. These coordinates were determined by examining the pixel axes in Cell 2 and identifying where the tracks intersect the road surface on both sides. OpenCV's polylines function draws the polygon as a green outline, and putText adds a readable label. The copy() call creates a duplicate of the frame so the original pixel data is not permanently modified. This manually-defined polygon is the starting point that later cells dynamically stabilize.

Cell 4 — Detect Background Feature Points for Optical Flow

Code:

```
# Cell 4 - Detect Background Feature Points for Optical Flow
cap = cv2.VideoCapture(video_path)
ret, frame = cap.read()
cap.release()

# Convert to grayscale — optical flow works on grayscale
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Detect strong corner points in the background to track
# These are stable reference points the system will follow frame to frame
feature_params = dict(
```

```
    maxCorners=200,        # max number of points to track
    qualityLevel=0.01,     # minimum quality of points
    minDistance=30,        # minimum distance between points
    blockSize=7            # size of area considered around each point
)

feature_points = cv2.goodFeaturesToTrack(gray, **feature_params)

# Draw the detected points on the frame
frame_features = frame.copy()
for point in feature_points:
    x, y = point.ravel().astype(int)
    cv2.circle(frame_features, (x, y), 8, (0, 0, 255), -1)  # red dots

# Also draw the ZOI so we can see both together
cv2.polylines(frame_features, [zoi_points], isClosed=True, color=(0, 255, 0),
thickness=5)

# Display
frame_rgb = cv2.cvtColor(frame_features, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(14, 8))
plt.imshow(frame_rgb)
plt.title(f"Feature Points Detected: {len(feature_points)}")
plt.axis("off")
plt.show()
```

Explanation:

Cell 4 detects strong feature points in the first frame that will serve as stable background reference markers for tracking camera movement. OpenCV's goodFeaturesToTrack function identifies corners and edges in the image — locations where pixel intensity changes significantly in multiple directions, making them easy to locate and follow frame to frame. The parameters control how many points to find (maxCorners=200), the minimum corner strength required (qualityLevel), and how spread apart the points must be (minDistance). The frame is first converted to grayscale because optical flow algorithms operate on single-channel intensity values rather than full color. The detected points are drawn as red dots and land predominantly on trees, poles, signs, and crossing structures — all static background elements that are ideal for measuring camera movement.

Cell 5 — Lucas-Kanade Optical Flow (Frame 1 to Frame 2)

Code:

```
# Cell 5 - Lucas-Kanade Optical Flow between Frame 1 and Frame 2
cap = cv2.VideoCapture(video_path)

# Read frame 1
ret, frame1 = cap.read()
gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)

# Detect feature points on frame 1
feature_points = cv2.goodFeaturesToTrack(gray1, **feature_params)

# Read frame 2
ret, frame2 = cap.read()
```

```
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

cap.release()

# Lucas-Kanade optical flow parameters
lk_params = dict(
    winSize=(21, 21),     # size of search window per pyramid level
    maxLevel=3,           # number of pyramid levels
    criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 30, 0.01)
)

# Track points from frame 1 to frame 2
points2, status, error = cv2.calcOpticalFlowPyrLK(
    gray1, gray2, feature_points, None, **lk_params
)

# Filter only successfully tracked points
good_old = feature_points[status == 1]
good_new = points2[status == 1]

# Draw the movement lines and points
frame_flow = frame2.copy()
for old, new in zip(good_old, good_new):
    x_old, y_old = old.ravel().astype(int)
    x_new, y_new = new.ravel().astype(int)
    cv2.line(frame_flow, (x_old, y_old), (x_new, y_new), (255, 255, 0), 2)  #
yellow line = movement
    cv2.circle(frame_flow, (x_new, y_new), 6, (0, 0, 255), -1)              # red
dot = new position

# Draw ZOI
cv2.polylines(frame_flow, [zoi_points], isClosed=True, color=(0, 255, 0),
thickness=5)

# Display
frame_rgb = cv2.cvtColor(frame_flow, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(14, 8))
plt.imshow(frame_rgb)
plt.title(f"Optical Flow — Frame 1 to Frame 2 | Tracked Points: {len(good_new)}")
plt.axis("off")
plt.show()

print(f"Points successfully tracked: {len(good_new)} / {len(feature_points)}")
```

Explanation:

Cell 5 runs the Lucas-Kanade optical flow algorithm, which tracks how each of the 200 feature points from Frame 1 moved to a new position in Frame 2. The algorithm works by analyzing a small window of pixels around each point and searching for the best matching position in the next frame using a pyramid-based approach — the image is analyzed at multiple scales (controlled by maxLevel) so both large and small movements can be detected reliably. The function returns the new point positions, a status array indicating whether each point was successfully tracked (1) or lost (0), and an error value. Yellow lines connect each point's old position to its new position showing movement direction and distance. Between two consecutive frames with a stable camera the lines are extremely short because the points barely move —

which is expected and confirms the camera is steady. During camera shake from a passing train, these lines would be visibly longer.

Cell 6 — RANSAC Homography & ZOI Stabilization

Code:

```
# Cell 6 - RANSAC Homography to Stabilize the ZOI
# Take the tracked points and compute a homography using RANSAC
H, mask_ransac = cv2.findHomography(
    good_old, good_new,
    cv2.RANSAC,
    ransacReprojThreshold=3.0  # max pixel error to be considered an inlier
)

# Separate inliers (stable background) from outliers (moving objects)
inliers = good_new[mask_ransac.ravel() == 1]
outliers = good_new[mask_ransac.ravel() == 0]

print(f"Total tracked points : {len(good_new)}")
print(f"Inliers  (background): {len(inliers)}")
print(f"Outliers (moving obj): {len(outliers)}")

# Warp the ZOI polygon using the homography to keep it locked to the tracks
zoi_float = zoi_points.astype(np.float32).reshape(-1, 1, 2)
zoi_stabilized = cv2.perspectiveTransform(zoi_float, H).reshape(-1,
2).astype(np.int32)

# Draw everything on frame 2
frame_ransac = frame2.copy()

# Original ZOI in green
cv2.polylines(frame_ransac, [zoi_points], isClosed=True, color=(0, 255, 0),
thickness=3)

# Stabilized ZOI in blue
cv2.polylines(frame_ransac, [zoi_stabilized], isClosed=True, color=(255, 0, 0),
thickness=3)

# Inlier points in green, outliers in red
for pt in inliers:
    cv2.circle(frame_ransac, tuple(pt.astype(int)), 6, (0, 255, 0), -1)
for pt in outliers:
    cv2.circle(frame_ransac, tuple(pt.astype(int)), 6, (0, 0, 255), -1)

# Legend
cv2.putText(frame_ransac, "Green = original ZOI", (50, 80),
            cv2.FONT_HERSHEY_SIMPLEX, 1.5, (0, 255, 0), 3)
cv2.putText(frame_ransac, "Blue = stabilized ZOI", (50, 140),
            cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255, 0, 0), 3)

frame_rgb = cv2.cvtColor(frame_ransac, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(14, 8))
plt.imshow(frame_rgb)
plt.title("RANSAC — Inliers vs Outliers | ZOI Stabilization")
plt.axis("off")
plt.show()
```

Explanation:

Cell 6 applies RANSAC (Random Sample Consensus) to compute a homography matrix — a mathematical transformation that describes how the camera moved between frames. RANSAC works by repeatedly sampling random subsets of tracked point pairs and finding the transformation that explains the most points consistently. Points that fit the transformation within the ransacReprojThreshold pixel tolerance are called inliers (stable background) while those that do not fit are outliers (potentially moving objects). Once the homography matrix H is computed from the stable inlier points, OpenCV's perspectiveTransform function uses it to warp the ZOI polygon coordinates — mathematically adjusting its corners to compensate for camera movement. The original ZOI is drawn in green and the stabilized version in blue, with the difference between the two showing exactly how much correction was applied. All 200 points being inliers confirms a stable, motionless scene between those two frames.

Cell 7 — Full Video Pipeline (Anchored to Frame 1)

Code:

```
# Cell 7 - Full Video Loop with ZOI Stabilization (Anchored to Frame 1)
cap = cv2.VideoCapture(video_path)

# Read and store the first frame as our permanent anchor
ret, first_frame = cap.read()
first_gray = cv2.cvtColor(first_frame, cv2.COLOR_BGR2GRAY)
first_points = cv2.goodFeaturesToTrack(first_gray, **feature_params)

# Reset to beginning
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
ret, prev_frame = cap.read()
prev_gray = first_gray.copy()

fps = cap.get(cv2.CAP_PROP_FPS)
frame_count = 0
snapshots = {}

capture_times = {
    "0s  — Static crossing"          :  0,
    "7s  — SUV trapped on tracks"    :  7,
    "14s — Gate damaged"             : 14,
    "28s — Train entering frame"     : 28
}

print("Processing video...")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_count += 1
    current_time = frame_count / fps
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Always track from FIRST frame points to CURRENT frame
    curr_points, status, _ = cv2.calcOpticalFlowPyrLK(
        first_gray, gray, first_points, None, **lk_params
```

```
    )

    # Filter valid points
    good_first = first_points[status == 1]
    good_curr  = curr_points[status == 1]

    H = None
    inlier_count = 0

    if len(good_first) >= 4:
        # Homography from first frame to current frame directly
        H, mask_ransac = cv2.findHomography(
            good_first, good_curr,
            cv2.RANSAC,
            ransacReprojThreshold=3.0
        )
        if mask_ransac is not None:
            inlier_count = int(mask_ransac.sum())

    # Always warp from ORIGINAL zoi_points using the direct homography
    frame_drawn = frame.copy()
    if H is not None:
        zoi_reshaped = zoi_points.astype(np.float32).reshape(-1, 1, 2)
        zoi_stabilized = cv2.perspectiveTransform(zoi_reshaped, H).reshape(-1,
2).astype(np.int32)
        cv2.polylines(frame_drawn, [zoi_stabilized], isClosed=True, color=(0, 255,
0), thickness=5)
    else:
        # Fallback to original if homography fails
        cv2.polylines(frame_drawn, [zoi_points], isClosed=True, color=(0, 0, 255),
thickness=5)

    cv2.putText(frame_drawn,
                f"Frame: {frame_count} | Time: {current_time:.1f}s | Inliers:
{inlier_count}",
                (50, 80), cv2.FONT_HERSHEY_SIMPLEX, 1.5, (0, 255, 0), 3)

    # Capture snapshots
    for label, target_sec in capture_times.items():
        if abs(current_time - target_sec) < (1 / fps) + 0.1:
            if label not in snapshots:
                snapshots[label] = cv2.cvtColor(frame_drawn, cv2.COLOR_BGR2RGB)
                print(f"  Captured: {label}")

cap.release()
print(f"\nDone! Processed {frame_count} frames.")

# Display snapshots
fig, axes = plt.subplots(2, 2, figsize=(18, 10))
axes = axes.ravel()

for i, (label, img) in enumerate(snapshots.items()):
    axes[i].imshow(img)
    axes[i].set_title(label, fontsize=12)
    axes[i].axis("off")

plt.suptitle("ZOI Stabilization — Fixed (Anchored to Frame 1)", fontsize=16)
plt.tight_layout()
plt.show()
```

Explanation:

Cell 7 runs the complete stabilization pipeline across every frame in the video. The critical design choice here is that the homography is always computed from the first frame directly to the current frame, rather than from the previous frame to the current frame. This anchor-to-frame-one approach eliminates cumulative drift — where small errors in each frame's transformation would compound over thousands of frames, causing the ZOI to gradually wander away from the tracks. For each frame, the system tracks where the original first-frame feature points have moved in the current frame, computes a RANSAC homography from those point pairs, and applies it directly to the original zoi_points to get the correctly stabilized polygon position. The inlier count on each frame shows how many stable background reference points were found — during the train passage this count drops slightly as the train temporarily blocks some reference points, but RANSAC uses the remaining visible inliers to maintain accurate stabilization. Snapshot frames at four key moments confirm the ZOI stays correctly positioned throughout the entire crossing event.

Cell 8 — MOG2 Background Subtraction Setup

Code:

```
# Cell 8 - MOG2 Background Subtraction Setup
# MOG2 learns what the "empty" crossing looks like from the first N frames
# After training, anything new that appears gets flagged as an anomaly

# Create the MOG2 background subtractor
mog2 = cv2.createBackgroundSubtractorMOG2(
    history=300,        # how many frames to use to learn the background
    varThreshold=50,    # how sensitive it is — higher = less sensitive
    detectShadows=True  # detect and mark shadows separately
)

# Train MOG2 on the first 300 frames (the static empty crossing)
cap = cv2.VideoCapture(video_path)
fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
frame_count = 0
print("Training MOG2 on background...")
while frame_count < 300:
    ret, frame = cap.read()
    if not ret:
        break
    mog2.apply(frame)
    frame_count += 1
print(f"MOG2 trained on {frame_count} frames.")

# Jump to test frame dynamically using seconds instead of hardcoded frame number
test_time = 14  # seconds — change this to test different moments
test_frame_num = min(int(test_time * fps), total_frames - 1)
cap.set(cv2.CAP_PROP_POS_FRAMES, test_frame_num)
ret, test_frame = cap.read()
cap.release()

# Apply MOG2 to the test frame — returns a mask
fg_mask = mog2.apply(test_frame)

# Display the original frame and the mask side by side
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
axes[0].imshow(cv2.cvtColor(test_frame, cv2.COLOR_BGR2RGB))
axes[0].set_title(f"Original Frame (~{test_time}s)")
axes[0].axis("off")
axes[1].imshow(fg_mask, cmap="gray")
axes[1].set_title("MOG2 Foreground Mask")
axes[1].axis("off")
plt.suptitle("MOG2 Background Subtraction", fontsize=14)
plt.tight_layout()
plt.show()
print(f"Mask shape: {fg_mask.shape}")
print("White pixels = detected anomalies, Gray = shadows, Black = background")
```

Explanation:

Cell 8 sets up and trains the MOG2 (Mixture of Gaussians 2) background subtraction algorithm. MOG2 works by learning what the empty crossing looks like over a set number of frames — defined by the history parameter. It builds a statistical model of the background for every pixel in the frame. Once trained, when a new frame is passed through it, MOG2 compares each pixel against its learned background model. Pixels that deviate significantly from the expected background are flagged as foreground anomalies and appear white in the output mask. Gray pixels represent shadows, which MOG2 detects separately when detectShadows=True. The varThreshold controls sensitivity — a higher value means a pixel must change more dramatically before being flagged, reducing false detections from minor lighting changes. The key strength of MOG2 is that it does not need to know what an object is — it simply flags anything that was not there when it was trained.

Cell 9 — YOLO Object Detection on Test Frame

Code:

```
# Cell 9 - YOLO Object Detection on the Same Test Frame
from ultralytics import YOLO

# Load the YOLO model — using yolo11n (nano) for speed
model = YOLO("../model/yolo11n.pt")

# Run YOLO on the same test frame from Cell 8
results = model(test_frame, verbose=False)

# Draw bounding boxes on a copy of the frame
frame_yolo = test_frame.copy()
for result in results:
    for box in result.boxes:
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        class_name = model.names[int(box.cls[0])]
        confidence = float(box.conf[0])
        cv2.rectangle(frame_yolo, (x1, y1), (x2, y2), (255, 0, 0), 3)
        label = f"{class_name} {confidence:.2f}"
        cv2.putText(frame_yolo, label, (x1, y1 - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255, 0, 0), 3)

# Display original and YOLO result side by side
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
axes[0].imshow(cv2.cvtColor(test_frame, cv2.COLOR_BGR2RGB))
```

```
axes[0].set_title("Original Frame (~14s)")
axes[0].axis("off")
axes[1].imshow(cv2.cvtColor(frame_yolo, cv2.COLOR_BGR2RGB))
axes[1].set_title("YOLO Detections")
axes[1].axis("off")
plt.suptitle("YOLO Object Detection", fontsize=14)
plt.tight_layout()
plt.show()

# Print detections
print("Objects detected by YOLO:")
for result in results:
    for box in result.boxes:
        class_name = model.names[int(box.cls[0])]
        confidence = float(box.conf[0])
        print(f"  - {class_name}: {confidence:.2f} confidence")
```

Explanation:

Cell 9 runs YOLOv11 on the same test frame used in Cell 8 to identify any known objects in the scene. YOLO (You Only Look Once) is a pre-trained deep learning model that scans the entire frame in a single pass and returns bounding boxes around anything it was trained to recognize — vehicles, people, bicycles, and other common objects. Each detection comes with a class name and a confidence score between 0 and 1. On the stress test video at 14 seconds, the model detected the white SUV as a truck with 0.60 confidence and a partially visible vehicle at the frame edge as a car with 0.35 confidence. The misclassification of the SUV as a truck rather than a car is not a problem for the pipeline — what matters is that YOLO identifies it as a known vehicle object so its bounding box can be subtracted from the MOG2 mask in the next step. The model weights are loaded from the model/ folder using a relative path (../model/yolo11n.pt) to keep project files organized.

Cell 10 — Active Exclusion Masking (MOG2 minus YOLO)

Code:

```
# Cell 10 - Active Exclusion Masking (MOG2 - YOLO = Unknown Objects)

# Step 1: Clean up the MOG2 mask
_, fg_mask_clean = cv2.threshold(fg_mask, 200, 255, cv2.THRESH_BINARY)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
fg_mask_clean = cv2.morphologyEx(fg_mask_clean, cv2.MORPH_OPEN,  kernel)
fg_mask_clean = cv2.morphologyEx(fg_mask_clean, cv2.MORPH_CLOSE, kernel)

# Step 2: Build ZOI mask and check vehicles using helper functions
zoi_mask = build_zoi_mask(test_frame.shape, zoi_points)
vehicle_in_zoi, vehicle_labels, exclusion_mask = check_vehicle_in_zoi(results,
zoi_mask, test_frame.shape)

# Step 3: Get debris mask using helper function
debris_mask = get_debris_mask(fg_mask_clean, exclusion_mask, zoi_mask)

# Display all three steps
fig, axes = plt.subplots(1, 3, figsize=(20, 6))
axes[0].imshow(fg_mask_clean, cmap="gray")
axes[0].set_title("MOG2 Mask (Cleaned)")
```

```
axes[0].axis("off")
axes[1].imshow(exclusion_mask, cmap="gray")
axes[1].set_title("After YOLO Subtraction")
axes[1].axis("off")
axes[2].imshow(debris_mask, cmap="gray")
axes[2].set_title("Debris Mask (Inside ZOI Only)")
axes[2].axis("off")
plt.suptitle("Active Exclusion Masking Pipeline", fontsize=14)
plt.tight_layout()
plt.show()

# Count anomalous pixels inside ZOI
debris_pixel_count = cv2.countNonZero(debris_mask)
zoi_pixel_count    = cv2.countNonZero(zoi_mask)
print(f"Anomalous pixels inside ZOI after YOLO subtraction: {debris_pixel_count}")
print(f"ZOI total pixels: {zoi_pixel_count}")
print(f"Debris coverage: {(debris_pixel_count / zoi_pixel_count) * 100:.2f}%")
```

Explanation:

Cell 10 implements the active exclusion masking technique — the mathematical subtraction of YOLO bounding boxes from the MOG2 anomaly mask. The process happens in three steps, now using helper functions from pipeline_helpers.py for cleaner, reusable code. First the MOG2 mask is cleaned up by applying a binary threshold to remove gray shadow pixels, followed by morphological operations that remove tiny noise pixels (MORPH_OPEN) and fill small gaps (MORPH_CLOSE). Second, build_zoi_mask() creates the filled polygon mask for the ZOI, and check_vehicle_in_zoi() tests whether any YOLO bounding box overlaps the ZOI polygon — returning a boolean flag, the list of overlapping vehicle labels, and the exclusion mask. Third, get_debris_mask() subtracts the exclusion mask from the cleaned MOG2 mask and isolates only pixels inside the ZOI. The new vehicle_in_zoi flag is the key output of this cell — it feeds directly into the safety logic in Cell 11 to trigger an immediate ALARM when a known vehicle is detected inside the danger zone.

Cell 11 — Safety Logic: Area Threshold + Vehicle-in-ZOI Check

Code:

```
# Cell 11 - Safety Logic: Area Threshold and Debris Timer

# Safety thresholds
AREA_THRESHOLD    = 5000  # minimum pixel area for unknown debris
TIME_THRESHOLD    = 3.0   # seconds debris must persist before alarm
VEHICLE_THRESHOLD = 5.0   # seconds a known vehicle can stay in ZOI before alarm

# Count pixels
debris_pixel_count = cv2.countNonZero(debris_mask)
zoi_pixel_count    = cv2.countNonZero(zoi_mask)

# Evaluate status using helper functions (no timers on single frame)
status_text, status_color = evaluate_status(
    debris_time=0.0,
    vehicle_time=0.0,
    vehicle_in_zoi=vehicle_in_zoi,
    TIME_THRESHOLD=TIME_THRESHOLD,
    VEHICLE_THRESHOLD=VEHICLE_THRESHOLD
```

```
)

# Override if debris exceeds area threshold
if debris_pixel_count >= AREA_THRESHOLD:
    status_text  = "ALARM"
    status_color = (0, 0, 255)

# Print evaluation
print("=== Safety Logic Evaluation ===")
print(f"Anomalous pixels inside ZOI : {debris_pixel_count}")
print(f"Area threshold              : {AREA_THRESHOLD} pixels")
print(f"Time threshold (debris)     : {TIME_THRESHOLD} seconds")
print(f"Time threshold (vehicle)    : {VEHICLE_THRESHOLD} seconds")
print(f"Vehicle detected in ZOI     : {vehicle_in_zoi}")
if vehicle_in_zoi:
    print(f"Vehicle labels              : {', '.join(vehicle_labels)}")
print()
print(f"STATUS: {status_text}")
print(f"Debris coverage: {(debris_pixel_count / zoi_pixel_count) * 100:.2f}%")

# Visualize using draw_frame helper
frame_safety = draw_frame(
    frame=test_frame,
    zoi_stabilized=zoi_points,
    yolo_results=results,
    zoi_mask=zoi_mask,
    status_text=status_text,
    status_color=status_color,
    frame_count=0,
    current_time=14.0,
    debris_count=debris_pixel_count,
    vehicle_in_zoi=vehicle_in_zoi,
    vehicle_time=0.0
)

plt.figure(figsize=(14, 8))
plt.imshow(cv2.cvtColor(frame_safety, cv2.COLOR_BGR2RGB))
plt.title("Safety Logic Evaluation — ZOI Debris + Vehicle Detection")
plt.axis("off")
plt.show()
```

Explanation:

Cell 11 implements the safety logic with two independent detection paths. The first path is the existing debris pixel count: if unknown anomalous pixels exceed AREA_THRESHOLD (5,000 pixels) inside the ZOI, the status moves toward ALARM. The second path — added in response to professor feedback and the stress test findings — is the Vehicle-in-ZOI check: if check_vehicle_in_zoi() returned True from Cell 10, meaning a YOLO-detected vehicle's bounding box overlaps the ZOI polygon, evaluate_status() returns WARNING immediately regardless of pixel count. This secondary check closes a critical gap identified during stress testing — the original debris-only logic would return CLEAR even when a vehicle was fully trapped on the tracks, because YOLO correctly excluded the vehicle from the debris count, leaving too few pixels to trigger an alarm. The draw_frame() helper handles all visual overlays — ZOI polygon, YOLO bounding boxes with "IN ZOI" labels when applicable, status text, and the frame info bar — keeping the visualization code out of the main logic. These thresholds are

tunable parameters that would be calibrated based on the physical size of the crossing and camera resolution in a real deployment.

Cell 12 — Full Pipeline: ZOI + MOG2 + YOLO + Vehicle-in-ZOI Safety Logic

Code:

```
# Cell 12 - Full Pipeline: ZOI + MOG2 + YOLO + Safety Logic (with Vehicle Timer)
# Safety thresholds
AREA_THRESHOLD    = 5000
TIME_THRESHOLD    = 3.0
VEHICLE_THRESHOLD = 3.0

# Train MOG2
cap = cv2.VideoCapture(video_path)
fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
mog2 = cv2.createBackgroundSubtractorMOG2(history=300, varThreshold=50,
detectShadows=True)
frame_count = 0
print("Training MOG2...")
while frame_count < 300:
    ret, frame = cap.read()
    if not ret:
        break
    mog2.apply(frame)
    frame_count += 1
print("MOG2 ready.")

# Reset to beginning
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

# Timers and tracking
debris_timer_start  = None
vehicle_timer_start = None
frame_count = 0
snapshots   = {}

capture_times = {
    "0s  — SUV entering as gates come down" :  0,
    "7s  — SUV fully trapped on tracks"     :  7,
    "14s — Gate damaged, SUV still on tracks": 14,
    "28s — Train entering frame"            : 28
}

print("Processing full pipeline...")
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame_count += 1
    current_time = frame_count / fps

    # ZOI Stabilization
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    zoi_stabilized = stabilize_zoi(gray, first_gray, first_points, lk_params,
zoi_points)
    zoi_mask_full  = np.zeros(frame.shape[:2], dtype=np.uint8)
```

```
    cv2.fillPoly(zoi_mask_full, [zoi_stabilized], 255)

    # MOG2
    fg_mask = apply_mog2_mask(frame, mog2)
    if frame_count == 1:
        yolo_results   = []
        vehicle_in_zoi = False
        vehicle_labels = []
        exclusion_mask = np.zeros(frame.shape[:2], dtype=np.uint8)

    # YOLO + Vehicle Check (every 5 frames for speed)
    if frame_count % 5 == 0:
        yolo_results = model(frame, verbose=False)
        vehicle_in_zoi, vehicle_labels, exclusion_mask = check_vehicle_in_zoi(
            yolo_results, zoi_mask_full, frame.shape
        )
    # On skipped frames, keep the last known YOLO results
    else:
        pass

    # Debris Mask
    debris_mask  = get_debris_mask(fg_mask, exclusion_mask, zoi_mask_full)
    debris_count = cv2.countNonZero(debris_mask)

    # Vehicle Timer
    if vehicle_in_zoi:
        if vehicle_timer_start is None:
            vehicle_timer_start = current_time
        vehicle_time = current_time - vehicle_timer_start
    else:
        vehicle_timer_start = None
        vehicle_time = 0.0

    # Debris Timer
    if debris_count >= AREA_THRESHOLD:
        if debris_timer_start is None:
            debris_timer_start = current_time
        debris_time = current_time - debris_timer_start
    else:
        debris_timer_start = None
        debris_time = 0.0



    # Status — vehicle in ZOI triggers ALARM immediately
    if vehicle_in_zoi:
        status_text  = "ALARM"
        status_color = (0, 0, 255)
    elif debris_time >= TIME_THRESHOLD:
        status_text  = "ALARM"
        status_color = (0, 0, 255)
    else:
        status_text, status_color = evaluate_status(
            debris_time, vehicle_time, vehicle_in_zoi,
            TIME_THRESHOLD, VEHICLE_THRESHOLD
        )

    # Draw Frame
    frame_drawn = draw_frame(
        frame, zoi_stabilized, yolo_results, zoi_mask_full,
        status_text, status_color, frame_count, current_time,
        debris_count, vehicle_in_zoi, vehicle_time
```

```
    )

    # Snapshots
    for label, target_sec in capture_times.items():
        if abs(current_time - target_sec) < (1 / fps) + 0.1:
            if label not in snapshots:
                snapshots[label] = cv2.cvtColor(frame_drawn, cv2.COLOR_BGR2RGB)
                print(f"  Captured: {label}")

cap.release()
print(f"\nDone! Processed {frame_count} frames.")

# Display
fig, axes = plt.subplots(2, 2, figsize=(18, 10))
axes = axes.ravel()
for i, (label, img) in enumerate(snapshots.items()):
    axes[i].imshow(img)
    axes[i].set_title(label, fontsize=12)
    axes[i].axis("off")
plt.suptitle("Full Pipeline — ZOI + MOG2 + YOLO + Safety Logic", fontsize=16)
plt.tight_layout()
plt.show()
```

Explanation:

Cell 12 combines every module into a single end-to-end pipeline that processes the entire video frame by frame. For each frame the system runs six layers in sequence. First, ZOI stabilization anchors the danger zone polygon to the physical tracks using optical flow and RANSAC homography always referenced from the first frame to prevent cumulative drift. Second, MOG2 generates a foreground mask of anything that changed from the learned background. Third, YOLO runs every 5 frames for performance — on skipped frames the last known YOLO results are preserved so vehicle detection stays continuous. YOLO variables (yolo_results, vehicle_in_zoi, exclusion_mask) are initialized at frame 1 to avoid NameErrors on the first skipped frames. Fourth, the active exclusion mask subtracts YOLO bounding boxes from the MOG2 mask and isolates debris inside the stabilized ZOI. Fifth, dual timers track how long debris has persisted and how long a known vehicle has been inside the ZOI. Sixth, the safety logic applies a direct priority check — if vehicle_in_zoi is True, STATUS immediately becomes ALARM regardless of debris pixel count or timer values. If debris_time exceeds TIME_THRESHOLD (3.0 seconds) the system also triggers ALARM. Otherwise evaluate_status() handles the CLEAR and WARNING states. On the stress test video (SUV trapped on tracks), this pipeline correctly outputs CLEAR at 0s (empty crossing), ALARM at 7s (SUV fully trapped), ALARM at 14s (SUV still in ZOI, gate arm damaged), and ALARM at 28s (train approaching, SUV still in ZOI).